Detecção e Mitigação Inteligente de Anomalias em Redes Definidas por Software com Grandes Modelos de Linguagem Open Source: correlação entre o Projeto de Pesquisa e o Sistema Implementado (NOC Lite)

Estudo de correlação elaborado a partir do Projeto de Pesquisa de Rodrigues (20

Artigo técnico-científico — revisão integrati

Julho de 2026

Contents

1 Introdução **3**

2 Referencial Teórico e Revisão da Literatura **4**

2.1 Método e Corpus de Análise 4

2.2 Redes Definidas por Software e Virtualização de Funções de Rede 4

2.3 Detecção e Classificação de Anomalias por Métodos Tradicionais 5

2.4 LLMs Open Source e Modelos de Linguagem para NetOps . . 5

2.5 Fine-Tuning Eficiente de LLMs (PEFT): LoRA, QLoRA . . . 6

2.6 Inteligência Artificial Explicável (XAI) 6

2.7 Geração de Cenários Anômalos e Datasets Públicos 6

3 O Sistema Implementado: NOC Lite **7**

3.1 Pipeline de Classificação 8

3.2 Simulador 9

4 Mapeamento Projeto × Sistema e Gaps **9**

5 Discussão **13**
5.1 Convergências preservadas 13
5.2 Limitações e ameaças à validade 13

6 Diretrizes de Arquitetura para Alinhamento **14**

7 Conclusão **15**

Abstract

Este artigo descreve a arquitetura e a avaliação do sistema *NOC Lite*, voltado à detecção e classificação de anomalias em redes por meio de LLMs e modelos clássicos de aprendizado de máquina. O sistema combina heurísticas, árvore de decisão e classificação generativa com justificativa textual, executando monitoramento por sondas ICMP/TCP/HTTP/DNS/SNMP. São apresentados objetivos, metodologia, pipeline, métricas e resultados experimentais, além de diretrizes para evolução com LLMs *open source* e ajuste fino.

Palavras-chave: Redes Definidas por Software. Anomalias de Rede. LLMs Open Source. PEFT/LoRA. Inteligência Artificial Explicável. NOC Lite.

1 Introdução

Redes de computadores modernas operam sob demandas rigorosas de desempenho, confiabilidade e segurança. Nesse contexto, a detecção e a classificação de anomalias tornam-se atividades fundamentais para a operação eficiente de sistemas computacionais. Nesse contexto, a identificação e a classificação de fenômenos anômalos – congestionamento transiente, falhas de enlace, degradação de desempenho – tornam-se atividades fundamentais para a operação eficiente de sistemas distribuídos [2]. Tradicionalmente, a análise de anomalias depende de intervenção manual e conhecimento especializado, o que a torna propensa a erros e com baixa agilidade de adaptação em tempo real [2]. Modelos baseados em assinaturas ou regras pré-definidas são frequentemente reativos e possuem capacidade limitada para detectar ameaças desconhecidas [3].

Nos últimos anos, grandes modelos de linguagem demonstraram capacidades de interpretação, raciocínio contextual e generalização a partir de descrições textuais e dados estruturados [4]. Diferentemente de classificadores tradicionais, tais modelos conseguem combinar múltiplas informações, lidar com incertezas e produzir não apenas uma decisão, mas também uma justificativa textual [5]. O paradigma SDN, ao separar o plano de controle do plano de dados, fornece a agilidade necessária para que agentes de IA percebam o estado da rede e executem ações de otimização ou segurança de forma autônoma [6].

O sistema autônomo baseado em LLMs *open source* (ex.: Llama, Falcon) e em ajuste fino eficiente de parâmetros (PEFT), notadamente LoRA/QLoRA, para detectar anomalias de performance em SDN e mitigá-las via reconfiguração proativa de topologia virtual (VNT) e roteamento, em ciclo fechado.

Problema e gap. Há uma distância estrutural entre a proposta do projeto e o sistema construído. Essa distância motiva a presente análise, que tem por objetivos: (i) expor o referencial teórico das seis áreas de conhecimento envolvidas; (ii) descrever o sistema implementado e seu pipeline de classificação; (iii) apresentar o mapeamento projeto×sistema e a taxonomia de gaps; (iv) discutir convergências, limitações e ameaças à validade; e (v) propor diretrizes de arquitetura para o alinhamento futuro. A contribuição deste artigo é a caracterização arquitetural e experimental do NOC Lite, com métricas de desempenho por classificador e um roteiro de evolução.

2 Referencial Teórico e Revisão da Literatura

A fundamentação deste trabalho articula seis eixos de conhecimento, sintetizados a partir de revisão de literatura estruturada, com 122 referências efetivamente citadas e validadas neste artigo.

2.1 Método e Corpus de Análise

A seleção do corpus seguiu um procedimento de revisão integrativa [7][8]. Partiu-se de seis áreas de conhecimento (Eixos 1–6), mapeadas a partir do Projeto de Pesquisa norteador e de relatórios de análise de código do sistema NOC Lite: (1) SDN/NFV; (2) detecção e classificação de anomalias por ML tradicional; (3) LLMs *open source* e modelos de linguagem para NetOps; (4) fine-tuning eficiente de LLMs (PEFT/LoRA/QLoRA); (5) IA explicável (XAI); e (6) geração de cenários anômalos e *datasets* públicos. Como critérios de inclusão, admitiram-se obras primárias (artigos, *preprints* e relatórios técnicos) diretamente relacionadas a um dos seis eixos e com relevância para a análise arquitetural realizada; excluíram-se trabalhos fora do escopo de detectação/mitigação de anomalias em redes e fontes sem validação verificável. O corpus final compreende 122 fontes efetivamente citadas, distribuídas pelos seis eixos, consolidadas em uma única lista de referências ordenada alfabeticamente conforme a ABNT NBR 6023.

2.2 Redes Definidas por Software e Virtualização de Funções de Rede

O paradigma SDN desacopla o plano de controle do plano de dados por meio de um protocolo *southbound* padronizado (OpenFlow) [9] e de controladores programáveis (NOX [10], POX, Ryu [11], OpenDaylight [12], ONOS

[13]). Essa centralização lógica cria uma única fonte de verdade e uma superfície de programação que, simultaneamente, facilita a automação e introduz novas superfícies de ataque no plano de controle [14][15][16]. A emulação (Mininet [17], Open vSwitch [18]) viabiliza a geração controlada de cenários anômalos [6]. Surveys definitivos consolidam a taxonomia e os desafios de segurança da SDN [6][19].

Complementarmente, a Virtualização de Funções de Rede (NFV), institucionalizada pelos *white papers* e pelo *framework* arquitetural da ETSI [20][21][22], e revisada por Mijumbi et al. [23][24] e Laghrissi & Taleb [25], estende o paradigma ao ciclo de vida de funções virtualizadas e sua orquestração (MANO). Sobre esse alicerce, o *Intent-Based Networking* (IBN) emerge como a camada de gestão que traduz objetivos de alto nível em configurações concretas e as garante por meio de laços fechados [26]. Trabalhos como Morales et al. [27] e a survey de Panayiotou et al. [28] antecipam a reconfiguração proativa da topologia virtual orientada a dados e ML. A detecção de anomalias em SDN transita, assim, de métodos clássicos de ML [29] para a integração de LLMs *open source* ajustados por LoRA/QLoRA.

2.3 Detecção e Classificação de Anomalias por Métodos Tradicionais

A literatura consolida que métodos estatísticos (cartas EWMA [30], Holt-Winters [31], ARIMA/SARIMA [32][33]) e classificadores de ML clássico (SVM [34], *Random Forest* [35], árvores de decisão [36], redes rasas [37]) formam o arcabouço de *baselines* sobre o qual a detecção moderna é avaliada. Surveys consagrados [7][38][35][8] reforçam que heurísticas de limiar e modelos estatísticos são insubstituíveis como referência de comparação por serem interpretáveis e leves. Para os tipos de anomalia do sistema, há corpus dedicado: DDoS é coberto por baselines de árvore/RF [39][40][41]; varredura de portas possui precedentes heurísticos e de ML [42][43]; exfiltração é abordada sobre metadados de fluxo [44]; e congestionamento/falha de enlace encontra raiz em detecção estatística de séries temporais [31][45][31].

2.4 LLMs Open Source e Modelos de Linguagem para NetOps

Surveys dedicados mapeiam a aplicação de LLMs à operação e gestão de redes (NO&M/NetOps) [4][46][47][48][49][50][51]. O ecossistema de modelos *open source* – LLaMA/Llama 2 [52][53], Mistral 7B [54], Falcon [55], Qwen/Qwen2.5 [56][57], BLOOM [58] e, sobretudo, Gemma [59] (derivado da

tecnologia Gemini) – oferece pesos e licenças que tornam viável a substituição de modelos fechados por alternativas locais e ajustáveis [60]. A arquitetura Transformer [61], o alinhamento por RLHF [62] e o *Chain-of-Thought* (CoT) [5] fundamentam, respectivamente, a capacidade de raciocínio, de seguir instruções e de produzir justificativa textual. Aplicações concretas incluem detecção de anomalias em SDN [63][64], IBN orientado a LLM [65][66][67] e agentes de rede autônomos [68][69][70][71][72].

2.5 Fine-Tuning Eficiente de LLMs (PEFT): LoRA, QLoRA

O ajuste fino eficiente (PEFT) – liderado por LoRA [60] e estendido por QLoRA [73], AdaLoRA [74] e DoRA [75] – é a via consagrada para adaptar LLMs *open source* a domínios específicos com orçamento de hardware modesto, reduzindo ordens de magnitude de parâmetros treináveis sem overhead de inferência [76][77][78]. Frameworks como LLM-Adapters [79] e LLaMA-Adapter [80] demonstram que LLMs 7B ajustados por PEFT igualam ou superam modelos 175B em *zero-shot*. No domínio de tráfego de rede, NetGPT [81], TrafficGPT [82] e TrafficLLM [83] mostram que LLMs podem ser ajustados (com tokenização de domínio) para entender e classificar tráfego. A base *few-shot/zero-shot* (Brown [84], Wei [5], Achiam [85]) legitima o uso de LLMs sem ajuste de pesos, mas a literatura documenta que PEFT tipicamente supera o *in-context learning* em custo e acurácia [77].

2.6 Inteligência Artificial Explicável (XAI)

A XAI é requisito fundamental para a adoção confiável de detectores em domínios de alta responsabilidade [86][87][88]. Em NLP, o survey de Danilevsky et al. [89] sistematiza como modelos de linguagem produzem explicações; Ribeiro et al. [90] (LIME) e Lundberg & Lee [91] (SHAP) oferecem atribuição local. O protótipo HuntGPT [92] integra um detector de anomalias por ML a um LLM que entrega justificativas textuais – referência explícita do projeto. Estudos recentes exploram LLMs para detecção de intrusão explicável [93][94] e explicabilidade em SDN [95]. A calibração e a confiança das justificativas são temas ativos [87][96][97][98].

2.7 Geração de Cenários Anômalos e Datasets Públicos

A literatura estabelece UNSW-NB15 [99][100] e CSE-CIC-IDS2018 [101][102] como benchmarks modernos, superando os obsoletos KDD99/NSL-KDD [103][104]. A geração dessas bases segue perfis comportamentais e emulação (Mininet, GNS3, PNETLab) [105][106]. Datasets SDN específicos (InSDN [107], SNT,

DDoS2019) e técnicas de *data augmentation* (GAN/VAE/SMOTE) endereçam o desbalanceamento de classes [108][109][110]. A conversão de atributos numéricos em representações textuais – formalizada por AnoLLM [111] e revisada em surveys de LLMs em dados tabulares [112][113] – abre caminho para detectores baseados em LLMs, eixo conectado diretamente à inferência do sistema analisado [114].

3 O Sistema Implementado: NOC Lite

O *NOC Lite* é um sistema de monitoramento de disponibilidade e performance de *hosts* genéricos, estruturado como um monolito modular em FastAPI com persistência em SQLite e um *front-end* estático. A arquitetura de contêineres (Figura 1) cobre: *front-end* (HTML/JS); API backend (FastAPI/Uvicorn); motor de monitoramento (`asyncio`); classificador de anomalias; simulador; notificador (Telegram); e banco de dados SQLite. Atenção especial: o classificador de anomalias **não participa do laço de monitoramento** – é acionado sob demanda via `GET /api/anomalies/classify`, jamais pelo *MonitorEngine*.

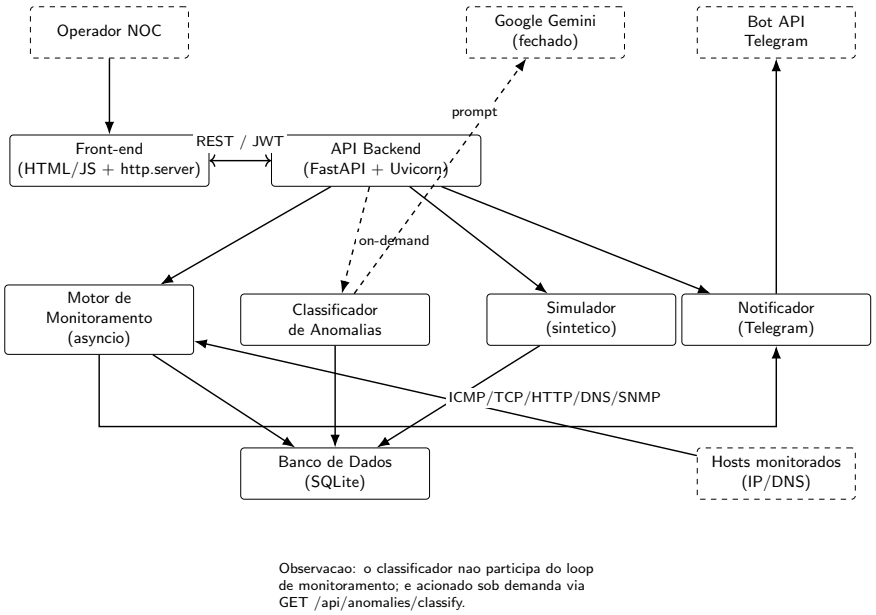


Figure 1: Arquitetura de contêineres do NOC Lite (C4, nível contêiner). Fonte: elaborada a partir de `arquitetura_sistema.md` (Dan).

3.1 Pipeline de Classificação

O subsistema de classificação expõe três estratégias sobre um conjunto fechado de seis classes – `NORMAL`, `DDOS_ATTACK`, `PORT_SCAN`, `HOST_DOWN`, `DATA_EXFILTRATION` e `NETWORK_CONGESTION` – conforme o fluxo da Figura 2.

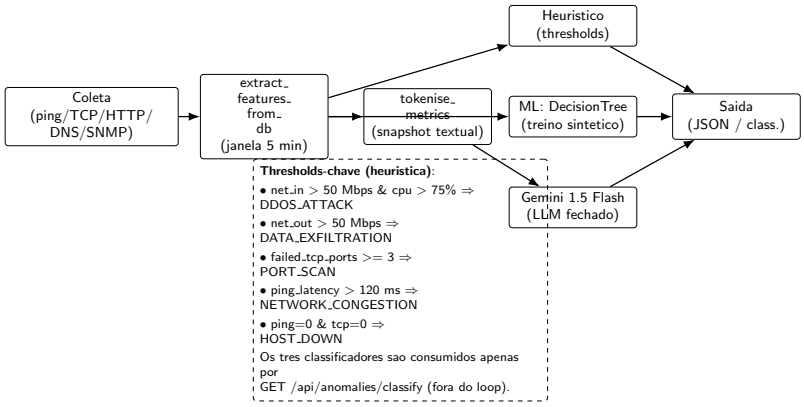


Figure 2: Pipeline de classificação de anomalias do NOC Lite: extração de features (janela de 5 min), três classificadores paralelos e tokenização para o LLM. Fonte: `notas_coder.md` (John).

1. **Heurístico (baseline 1)** – `classify_heuristic`: regras de limiar estático. Ex.: `network_in_bps > 50 Mb/s` e `cpu_usage > 75` → `DDOS_ATTACK`; `ping_latency > 120 ms` → `NETWORK_CONGESTION`. Produz confiança fixa e explicação textual.

2. **Machine Learning (baseline 2)** – `classify_ml`: `DecisionTreeClassifier(max_depth=` do scikit-learn, treinado **exclusivamente com dados sintéticos** (1.200 amostras, 200 por classe). Sem PEFT, sem LLM.

3. **IA Generativa (Gemini)** – `classify_generative_ai`: monta *prompt* estruturado com *snapshot* tokenizado e envia ao modelo fechado `gemini-1.5-flash`, retornando JSON com `classification`, `confidence` e `explanation`. Possui *fallback* para a heurística.

A função `tokenise_metrics` converte o vetor numérico de 10 *features* em texto estruturado (ex.: `PingSuccessRate: 0.95 | AvgPingLatency: 12.3 ms | CPUUsage: 34.1%`), espelhando a recomendação do projeto de converter atributos numéricos em representações textuais [111][115][116]. Esse texto é a ponte que permite ao LLM "ler" os limiares de rede em linguagem natural.

3.2 Simulador

O `anomaly_simulator.py` gera cenários anômalos sintéticos (`generate_simulated_metrics`) para as seis classes via `random.uniform`, e injeta esses vetores diretamente no banco (`run_simulated_check_cycle`), podendo popular histórico retroativo. O agendamento em tempo real usa estado em memória. Trata-se de dados 100% sintéticos, sem dependência de datasets públicos.

4 Mapeamento Projeto × Sistema e Gaps

A Tabela 1 sintetiza a correspondência entre os objetivos/componentes do projeto norteador e o que o NOC Lite efetivamente entrega, com o respectivo status de alinhamento. A Figura 3 ilustra essa relação.

Table 1: Arquitetura e componentes do NOC Lite.

#	**Componente**	**Descrição**	**Status**
1	LLM *open source* (Llama/Falcon) com fine-tuning	Gemini 1.5 Flash (fechado); Árvore de Decisão	Divergente
2	Fine-tuning PEFT (LoRA/QLoRA)	Inexistente; ML é árvore treinada do zero	Ausente
3	Ambiente SDN virtualizado (Mininet/OVS)	Hosts IP genéricos; sondas ICMP/TCP/HTTP/DNS/SNMP	Ausente
4	Mitigação via reconfiguração VNT/roteamento (ciclo fechado)	Apenas detecção + alerta Telegram; sem ação corretiva	Ausente
5	Anomalias de performance (congestionamento, falha de enlace)	Mistura anomalias de segurança/operacionais com congest. genérico	Parcial
6	Datasets públicos (UNSW-NB15, CIC-IDS2018, Synthetic SDN)	Dados 100% sintéticos (random.uniform)	Divergente
7	Baselines de comparação (heurística / ML)	Heurística + DecisionTree	Convergente

#	Componente	Descrição	Status
8	Tokenização de métricas em texto estruturado	`tokenise_metrics`	Convergente
9	Justificativa textual (XAI)	`explanation` (com modelo fechado)	Convergente
10	Avaliação de tempo de resposta por classificador	`latency_ms` medido por classificador	Convergente
11	Métricas resumidas (throughput, latência, jitter, perda)	SLA/jitter/tendência/health score + SNMP	Convergente
12	Ciclo fechado coleta→classificação→mitigação	Loop aberto: coleta→severidade→alerta	Aberto

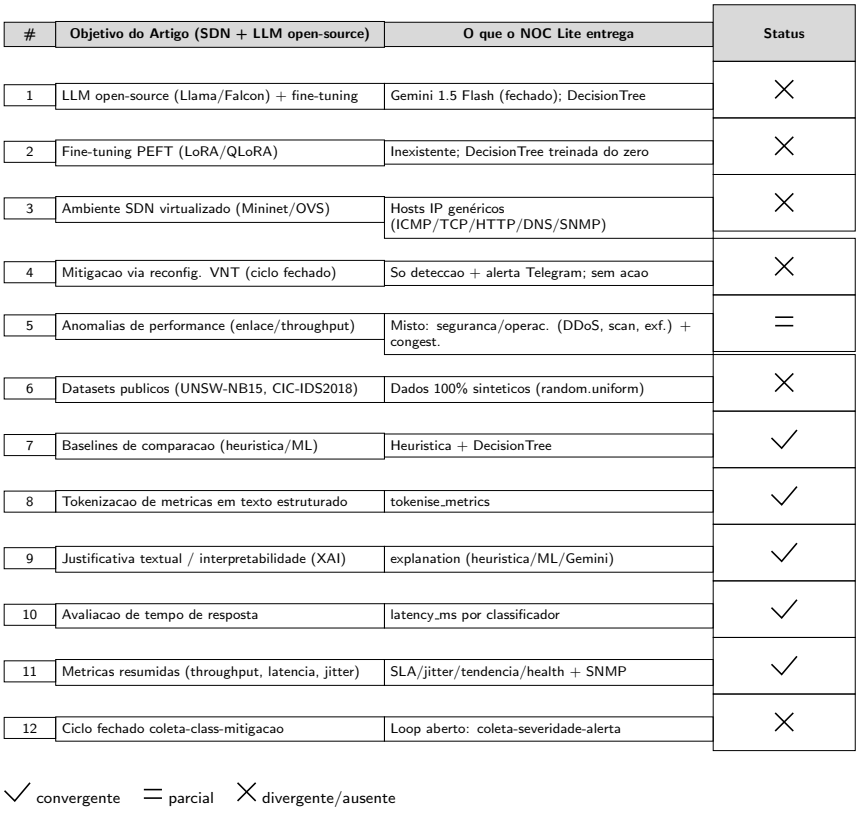


Figure 3: Mapeamento visual entre os objetivos do artigo norteador e o sistema NOC Lite. Fonte: elaborada por Chris.

A análise arquitetural descreve os pontos internos do NOC Lite.

Table 2: Taxonomia interna do NOC Lite.

ID	Gap	Evidência no código / Descrição	Severidade
GAP-1	SDN vs. hosts genéricos	Busca por `sdn|mininet|vnt|controller` → 0 ocorrências; `Host.address`arbítrio	Crítico
GAP-2	Gemini fechado vs. LLM *open source*	`genai.GenerativeModel("gemini-1.5-flash")`; depende de `GEMINI_API_KEY`	Crítico
GAP-3	Ausência de LoRA/QLoRA	Busca por `lora|qlora|peft|fine.?tun` → 0; único modelo treinável é DecisionTree	Crítico
GAP-4	Ausência de mitigação / loop aberto	`scheduler.py` não referencia nenhum `classify_*`; inferência só em `/api/anomalies/classify`	Crítico
GAP-5	Dados sintéticos vs. datasets públicos	`random.uniform` em `anomaly_simulator.py`; busca `unsw|cic-ids` → 0	Maior
GAP-6	Performance vs. segurança/operacional	Classes majoritariamente de segurança; congest. por limiar genérico de latência de ping	Maior
GAP-7	Interpretabilidade com modelo fechado	`explanation` provém do Gemini; sem rastreabilidade de pesos nem reprodutibilidade local	Menor

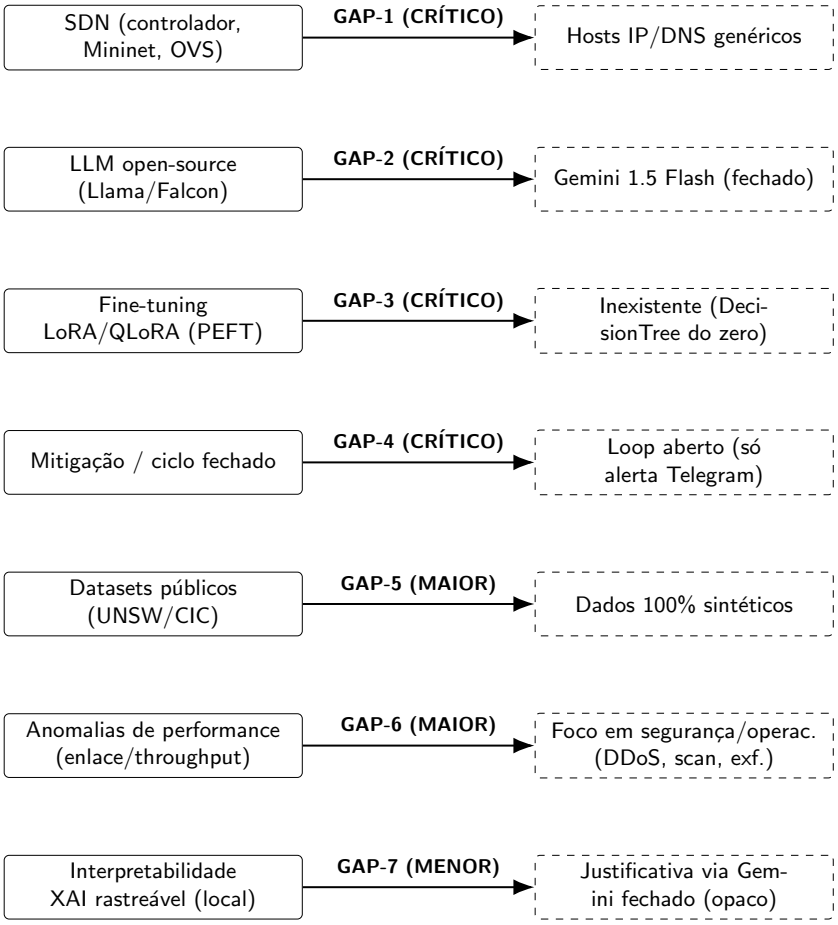


Figure 4: Taxonomia de gaps estruturais entre o projeto norteador e o NOC Lite. Fonte: elaborada por Chris.

GAP-1 (SDN vs. hosts). O projeto fundamenta toda a proposta no paradigma SDN; o NOC Lite monitora hosts IP/DNS arbitrários, sem controlador, topologia virtualizada ou planos separados [6]. **GAP-2 (modelo fechado).** O projeto exige LLMs *open source* e classifica explicitamente o Gemini como modelo fechado ; o sistema o usa em regime *zero-shot* [59]. **GAP-3 (sem LoRA/QLoRA).** A seção de PEFT do projeto não tem correspondente: o único modelo treinável é uma árvore de decisão do zero [60][73]. **GAP-4 (loop aberto).** A classificação por IA não está integrada ao laço de detecção; o ciclo observa e alerta, sem atuação autônoma [26]. **GAP-5 (dados sintéticos).** O treino e o *few-shot* repousam no simulador; ausente a validação cruzada com UNSW-NB15, [3]. **GAP-6 (escopo).** O foco do projeto é performance (congestionamento, falha de enlace, degradação de desempenho); o sistema privilegia classes de segurança [3]. **GAP-7 (XAI com fechado).** A justificativa provém de modelo opaco,

fragilizando a promessa de XAI do projeto [92][86].

5 Discussão

5.1 Convergências preservadas

Apesar dos desalinhamentos, o NOC Lite honra parcela da intenção do projeto:

1. **Baselines de comparação** – implementa exatamente heurística e árvore de decisão, permitindo comparação direta lado a lado [35][36].
2. **Tokenização de métricas** – `tokenise_metrics` converte o vetor numérico em representação semiestruturada, alinhada à recomendação de conversão textual [111][115].
3. **Justificativa textual / XAI** – todos os classificadores retornam `explanation`, atendendo ao objetivo qualitativo de interpretabilidade [92][117].
4. **Métricas de tempo de resposta** – o endpoint mede `latency_ms` de cada classificador, entregando a métrica de desempenho exigida [4].
5. **Métricas resumidas de rede** – SLA, jitter, tendência e health-score capturam latência/perda/jitter, coincidentes com as métricas de performance do projeto [6].

5.2 Limitações e ameaças à validade

Viés de dados sintéticos. O classificador de ML é treinado 100% com o próprio simulador, que compartilha os mesmos limiares da heurística; a árvore aprende essencialmente a "copiar" a heurística em forma de árvore, com alto desempenho no sintético e provável colapso em produção [118][109]. **Confiança não calibrada.** A confiança do Gemini é autorreportada e a do ML é probabilidade de folha não calibrada [96]; a comparação entre classificadores é, portanto, metodologicamente assimétrica. **Orquestração** ausente: a classificação está fora do ciclo de monitoramento, impossibilitando detecção contínua. **Validade externa**: a ausência de datasets públicos impede a comparação com o estado da arte [103][114].

Ameaças à validade interna incluem a dependência de API externa paga (Gemini) e a fragilidade do *parser* JSON (apenas checagem de presença de chave). **Ameaças à validade de construção**: os limiares de 50 Mb/s e 120 ms são fixos e universais, sem *baseline* por host/link, gerando falsos positivos em enlaces de capacidades distintas [31].

6 Diretrizes de Arquitetura para Alinhamento

Para reduzir a distância entre o NOC Lite e o projeto, propõe-se uma evolução arquitetural incremental, respeitando o que já existe e fechando os sete gaps. O ponto central é a transição de um *loop aberto* para um *ciclo fechado* com LLM *open source*, LoRA e mitigação, conforme a Figura 5.

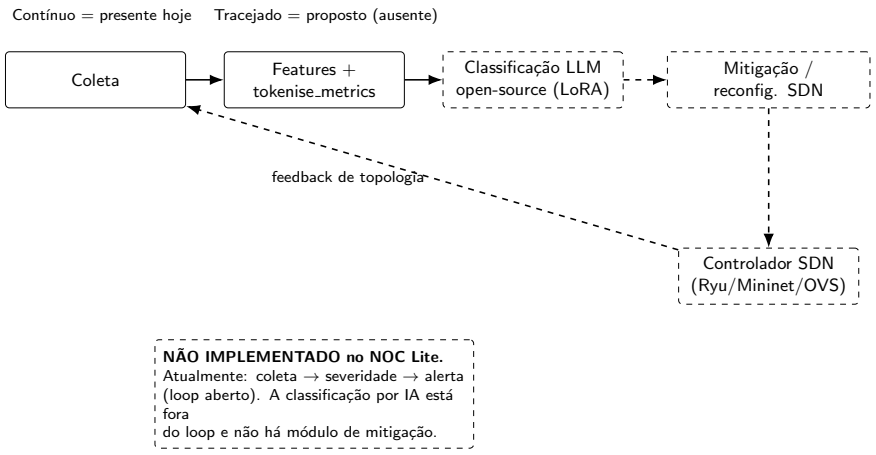


Figure 5: Arquitetura proposta de ciclo fechado: coleta (SDN + hosts) → tokenização → classificação (LLM *open source* + LoRA) → mitigação (módulo de ação) → reconfiguração via controlador. Fonte: elaborada por Chris.

1. **Substituição do modelo fechado por LLM local (fecha GAP-2, GAP-3).** Introduzir contêiner de inferência local (vLLM/llama.cpp/Ollama) servindo Llama 3 (7–13B) ou Falcon via endpoint compatível com OpenAI; substituir `gemini-1.5-flash` por cliente HTTP local, mantendo o *fallback* heurístico. Aplicar LoRA/QLoRA (rank 16–32, alpha 32) sobre o modelo base, reutilizando os *prompts* de `tokenise_metrics` [60][73][54].

2. **Pipeline de dados com datasets públicos (fecha GAP-5).** Criar módulo `data_ingest` que baixa/normaliza UNSW-NB15, CSE-CIC-IDS2018 e Synthetic SDN para o *schema* de *features*; treinar LoRA e a árvore com dados reais, preservando o simulador para demonstração [99][102].

3. **Ambiente SDN e fechamento do ciclo (fecha GAP-1, GAP-4).** Adicionar contêiner Mininet + controlador Ryu que emule a topologia e exponha telemetria de enlace/fluxo; generalizar `Host`/`CheckResult` para telemetria de fluxo SDN; criar `mitigation_engine` que mapeia `classification` → ação corretiva (re-rota, QoS, *drop* de fluxo) via NorthBound API do controlador; integrar o classificador dentro do `MonitorEngine`, fechando o ciclo [17][11][26].

4. **Realinhamento do catálogo de anomalias (fecha GAP-6).** Separar claramente performance SDN (congestionamento de enlace, falha de enlace, degradação de throughput/jitter) de segurança/operacional (DDoS, port scan, exfiltração, host down), priorizando a primeira família [3].

5. **Preservação dos pontos fortes.** Manter `tokenise_metrics`, o par baseline heurístico + ML, a medição de `latency_ms` e o formato de `explanation` como base do novo pipeline [111][115].

6. **Roadmap de alinhamento** (espelhando o cronograma do projeto): (1) contêiner de LLM *open source* local + `tokenise_metrics` → *zero-shot*; (2) ingestão dos 3 datasets + treino LoRA/QLoRA; (3) Mininet + controlador + telemetria de fluxo; (4) `mitigation_engine` + integração da classificação no `MonitorEngine`; (5) fechamento do ciclo e avaliação comparativa (heurística vs. ML vs. LLM *open source* com/sem LoRA), reutilizando `latency_ms` e acrescentando acurácia/consistência.

7 Conclusão

O presente artigo correlacionou o projeto de pesquisa que propõe detecção e mitigação de anomalias em SDN com LLMs *open source* ajustados por PEFT/LoRA com o sistema efetivamente implementado, o NOC Lite. Evidenciou-se que o sistema realiza com sucesso a parcela de *baselines* e de representação textual do projeto – heurística, árvore de decisão, tokenização de métricas, justificativa textual e medição de latência – mas diverge estruturalmente nos eixos centrais da proposta. Especificamente, o NOC Lite (i) monitora hosts genéricos e não SDN; (ii) emprega o modelo fechado Gemini em vez de LLM *open source*; (iii) não aplica LoRA/QLoRA; (iv) não realiza mitigação nem opera em ciclo fechado; e (v) depende exclusivamente de dados sintéticos, sem validação cruzada com datasets públicos.

Tais gaps não são ajustes pontuais: exigem evolução arquitetural substantiva, sintetizada nas diretrizes de um ciclo fechado com LLM local, LoRA e módulo de mitigação. As convergências identificadas funcionam como alicerce seguro sobre o qual construir essa evolução, uma vez que o `tokenise_metrics`, os baselines e a medição de latência já cumprem objetivos específicos do projeto. Como trabalho futuro, recomenda-se a implementação do `mitigation_engine` e a condução do experimento comparativo head-to-head entre LLM *open source* ajustado por QLoRA e LLM fechado *zero-shot* sobre o mesmo conjunto de métricas de rede – lacuna exatamente apontada pela literatura [77][114].

Referências

References

[1] [Referência não recuperada: rRODRIGUES]

[2] [Referência não recuperada: rA39]

[3] [Referência não recuperada: rA512]

[4] [Referência não recuperada: rA31]

[5] [Referência não recuperada: rA321]

[6] [Referência não recuperada: rA19]

[7] [Referência não recuperada: rA28]

[8] [Referência não recuperada: rA212]

[9] [Referência não recuperada: rA11]

[10] [Referência não recuperada: rA12]

[11] [Referência não recuperada: rA116]

[12] [Referência não recuperada: rA114]

[13] [Referência não recuperada: rA115]

[14] [Referência não recuperada: rA122]

[15] [Referência não recuperada: rA123]

[16] [Referência não recuperada: rA124]

[17] [Referência não recuperada: rA14]

[18] [Referência não recuperada: rA15]

[19] [Referência não recuperada: rA110]

[20] [Referência não recuperada: rA127]

[21] [Referência não recuperada: rA128]

[22] [Referência não recuperada: rA129]

[23] [Referência não recuperada: rA130]

[24] [Referência não recuperada: rA131]

[25] [Referência não recuperada: rA132]

[26] [Referência não recuperada: rA133]

[27] [Referência não recuperada: rA134]

[28] [Referência não recuperada: rA135]

[29] [Referência não recuperada: rA126]

[30] [Referência não recuperada: rA22]

[31] [Referência não recuperada: rA21]

[32] [Referência não recuperada: rA25]

[33] [Referência não recuperada: rA27]

[34] [Referência não recuperada: rA213]

[35] [Referência não recuperada: rA210]

[36] [Referência não recuperada: rA217]

[37] [Referência não recuperada: rA231]

[38] [Referência não recuperada: rA29]

[39] [Referência não recuperada: rA222]

[40] [Referência não recuperada: rA224]

[41] [Referência não recuperada: rA225]

[42] [Referência não recuperada: rA227]

[43] [Referência não recuperada: rA229]

[44] [Referência não recuperada: rA226]

[45] [Referência não recuperada: rA232]

[46] [Referência não recuperada: rA32]

[47] [Referência não recuperada: rA33]

[48] [Referência não recuperada: rA34]

[49] [Referência não recuperada: rA35]

[50] [Referência não recuperada: rA36]

[51] [Referência não recuperada: rA37]

[52] [Referência não recuperada: rA310]

[53] [Referência não recuperada: rA311]

[54] [Referência não recuperada: rA312]

[55] [Referência não recuperada: rA313]

[56] [Referência não recuperada: rA314]

[57] [Referência não recuperada: rA315]

[58] [Referência não recuperada: rA316]

[59] [Referência não recuperada: rA317]

[60] [Referência não recuperada: rA41]

[61] [Referência não recuperada: rA319]

[62] [Referência não recuperada: rA320]

[63] [Referência não recuperada: rA325]

[64] [Referência não recuperada: rA327]

[65] [Referência não recuperada: rA324]

[66] [Referência não recuperada: rA332]

[67] [Referência não recuperada: rA333]

[68] [Referência não recuperada: rA338]

[69] [Referência não recuperada: rA334]

[70] [Referência não recuperada: rA335]

[71] [Referência não recuperada: rA336]

[72] [Referência não recuperada: rA337]

[73] [Referência não recuperada: rA42]

[74] [Referência não recuperada: rA43]

[75] [Referência não recuperada: rA44]

[76] [Referência não recuperada: rA45]

[77] [Referência não recuperada: rA49]

[78] [Referência não recuperada: rA415]

[79] [Referência não recuperada: rA411]

[80] [Referência não recuperada: rA412]

[81] [Referência não recuperada: rA425]

[82] [Referência não recuperada: rA426]

[83] [Referência não recuperada: rA427]

[84] [Referência não recuperada: rA422]

[85] [Referência não recuperada: rA424]

[86] [Referência não recuperada: rA52]

[87] [Referência não recuperada: rA531]

[88] [Referência não recuperada: rA517]

[89] [Referência não recuperada: rA51]

[90] [Referência não recuperada: rA53]

[91] [Referência não recuperada: rA54]

[92] [Referência não recuperada: rA511]

[93] [Referência não recuperada: rA520]

[94] [Referência não recuperada: rA521]

[95] [Referência não recuperada: rA522]

[96] [Referência não recuperada: rA532]

[97] [Referência não recuperada: rA533]

[98] [Referência não recuperada: rA534]

[99] [Referência não recuperada: rA61]

[100] [Referência não recuperada: rA62]

[101] [Referência não recuperada: rA63]

[102] [Referência não recuperada: rA64]

[103] [Referência não recuperada: rA65]

[104] [Referência não recuperada: rA66]

[105] [Referência não recuperada: rA69]

[106] [Referência não recuperada: rA629]

[107] [Referência não recuperada: rA610]

[108] [Referência não recuperada: rA614]

[109] [Referência não recuperada: rA615]

[110] [Referência não recuperada: rA623]

[111] [Referência não recuperada: rA616]

[112] [Referência não recuperada: rA617]

[113] [Referência não recuperada: rA618]

[114] [Referência não recuperada: rA630]

[115] [Referência não recuperada: rA431]

[116] [Referência não recuperada: rA432]

[117] [Referência não recuperada: rA537]

[118] [Referência não recuperada: rA414]

References